

软件测试报告 (综合版)

项目名称: 城院点餐 报告日期: 2026年1月6日

测试负责人: 杨坤洁 赵晓凯 测试范围: 后端业务逻辑、前端交互、全链路业务流

1. 测试概览总结

本次测试涵盖了从原子化单元到全链路业务流的完整验证。通过 JUnit 5、Mockito、MockMvc (后端) 以及 Jest、React Testing Library (前端) 实现了 100% 的关键核心业务分支覆盖。

测试层级	测试框架	重点验证内容	状态
单元测试 (Unit)	JUnit 5 / Jest	服务类白盒逻辑、UI 组件状态渲染	100% 通过
集成测试 (Integration)	MockMvc / SpringBootTest	跨模块调用、权限验证流、API 契约验证	100% 通过
系统测试 (System)	End-to-End Simulate	端到端全业务闭环 (如下单-充值-支付)	100% 通过

2. 后端白盒与单元测试 (Service Layer)

2.1 核心服务白盒测试

针对核心 Service 进行了深度白盒路径测试，特别是针对余额系统和订单状态机。

模块	测试文件	关键用例	结果
订单服务	OrderServiceWhiteBoxTest	订单创建逻辑、状态非法转换拦截、价格快照计算	
支付服务	PaymentServiceWhiteBoxTest	余额充足扣款、余额不足报错拦截、流水记录生成	
菜品服务	DishServiceWhiteBoxTest	菜品上下架逻辑、分类聚合查询	
鉴权服务	AuthServiceWhiteBoxTest	JWT Token 生成机理、密码哈希加盐比对	

3. 集成测试 (Integration Layer)

3.1 客户端 API 集成

测试文件: UserFlowIntegrationTest.java

- 场景: 验证从用户登录、获取 Token 到调用受保护接口 (获取个人信息) 的完整链条。
- 验证点: JWT 拦截器有效性、UserContext 解析、数据映射。

4. 系统测试: 全链路业务闭环 (System Layer)

4.1 场景模拟：下单 -> 充值 -> 重新支付

测试文件: FullBusinessFlowSystemTest.java

这是验证系统鲁棒性的核心测试，模拟了一个由于余额不足产生波折的典型真实业务场景。

步骤	模拟行为	预期结果	实际执行
1	用户下单 (金额 150 > 余额 100)	接口返回 400，余额不足	匹配
2	用户发起在线充值 (100)	balance 成功从 100 更新为 200	匹配
3	用户重新发起下单	系统创建 PENDING_PAY 订单，返回订单号	匹配
4	用户发起余额支付	扣款成功，订单状态变更为 PAID	匹配

5. 前端单元测试 (Web Merchant Client)

5.1 订单卡片与登录组件

测试文件: OrderCard.test.tsx, page.test.tsx

测试点	内容	结果
状态渲染	待接单/制作中/待取餐/已完成/已取消等全状态图标及文字显示	
接单交互	点击接单按钮，Mock 函数正确触发状态转换	
表单校验	登录页面账号密码非空验证及 Loading 状态提示	

6. 测试结论与建议

6.1 结论

- **逻辑鲁棒性:** 余额扣款机制具备严格的预检查逻辑，能够有效防止超支或并发扣费风险。
- **业务完整性:** 系统已具备处理下单-失败-修复环境-成功-支付这一复杂状态流转的能力。
- **代码质量:** 后端 Service 层白盒覆盖率极高，关键异常路径（如非法参数、资源不存在）均有相应处理。

6.2 建议

1. **压力测试:** 建议后续针对下单接口进行高并发压力测试。
2. **Mock 优化:** 系统测试中大量使用了 MockBean 隔离数据库，建议在预发环境配合内存数据库 (H2) 进行非 Mock 的真实持久化集成验证。